



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sede Bogotá
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial

Ingeniería de Software I

Tutorial: Construcción de una API Backend con Python + Django

Docente:

Oscar Eduardo Alvarez Rodriguez

Alumnos:

Sergio Andrés Ramírez Céspedes

Juan Eduardo Garzon Rodriguez

Julian Andres Melo Bustos

Diego Phelipe Morales Aldana

Índice

1. Introducción	3
2. Dependencias principales	3
3. Configuración de la base de datos	3
3.1. Contenido de <code>docker-compose.yml</code>	3
3.2. Variables de entorno	4
4. Creación mínima de entidades	6
4.1. Vista “Hola Mundo” con conexión a la BD	6
5. Pasos para levantar el proyecto	6
6. Prueba final desde Postman	7

1. Introducción

Este tutorial muestra cómo construir una aplicación backend desde cero usando el mismo stack que tiene el proyecto abierto:

- Lenguaje: **Python**
- Framework: **Django 4.2**
- Biblioteca de API REST: **Django REST Framework 3.14**
- ORM: **Django ORM**
- Base de datos: **PostgreSQL**

2. Dependencias principales

El archivo `requirements.txt` usado en el proyecto contiene:

```
1 Django==4.2.0
2 djangorestframework==3.14.0
3 psycopg2-binary==2.9.12
4 python-decouple==3.8
5 django-cors-headers==4.0.0
```

3. Configuración de la base de datos

El proyecto usa Docker Compose para levantar PostgreSQL de forma rápida y aislada.

3.1. Contenido de `docker-compose.yml`

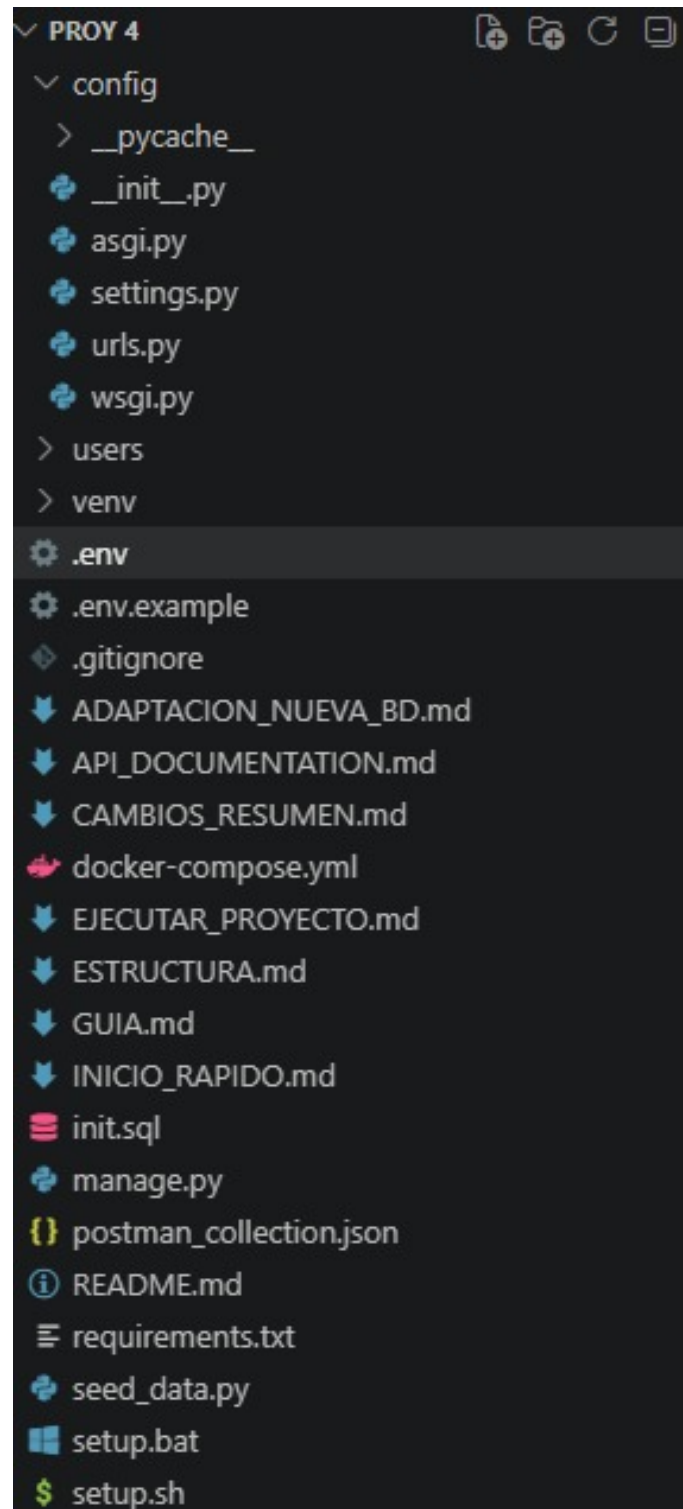
```
1 version: '3.8'
2
3 services:
4   db:
5     image: postgres:15
6     environment:
7       POSTGRES_DB: ingesoft1_db
8       POSTGRES_USER: ingesoft1_user
9       POSTGRES_PASSWORD: ingesoft123
10    ports:
11      - "5433:5432"
12    volumes:
```

```
13     - postgres_data:/var/lib/postgresql/data
14     - ./init.sql:/docker-entrypoint-initdb.d/init.sql
15 restart: unless-stopped
```

3.2. Variables de entorno

Crea un archivo `.env` en la raíz de tu proyecto:

```
1 SECRET_KEY=django-insecure-test-key
2 DEBUG=True
3 DATABASE_NAME=ingesoft1_db
4 DATABASE_USER=ingesoft1_user
5 DATABASE_PASSWORD=ingesoft123
6 DATABASE_HOST=localhost
7 DATABASE_PORT=5433
```

Figura 1: Estructura de archivos y ubicación del `.env`

4. Creación mínima de entidades

4.1. Vista “Hola Mundo” con conexión a la BD

Archivo: users/views.py

```
1 from rest_framework.decorators import api_view
2 from rest_framework.response import Response
3 from users.models import UserAccount, Role
4
5 @api_view(['GET'])
6 def hello_world(request):
7     role, _ = Role.objects.get_or_create(
8         name='User',
9         defaults={'description': 'Rol normal'}
10    )
11    user, created = UserAccount.objects.get_or_create(
12        email='hola@ejemplo.com',
13        defaults={
14            'name': 'Hola Mundo',
15            'password': '1234',
16        }
17    )
18    return Response({
19        'message': 'Hola Mundo desde Django + PostgreSQL',
20        'user_created': created,
21        'user_email': user.email
22    })
```

5. Pasos para levantar el proyecto

Nota Importante

Asegúrate de tener Docker corriendo en tu sistema antes de ejecutar el primer paso.

1. Iniciar Docker Compose:

```
1 docker-compose up -d
```

2. Iniciar el servidor:

```
1 python manage.py runserver
```

```
(venv) PS C:\Users\Juan\Desktop\Universidad\Semestre 7\Ingenieria de Software\Assignment 4\Proy 4> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
May 13, 2026 - 21:01:27
Django version 4.2, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Figura 2: Servidor de Django corriendo correctamente en la terminal

6. Prueba final desde Postman

1. Abre Postman.
2. Crea una nueva petición **GET**.
3. Ingresa la siguiente URL: `http://localhost:8000/api/nutrition/hello/`
4. Envía la petición presionando **Send**.

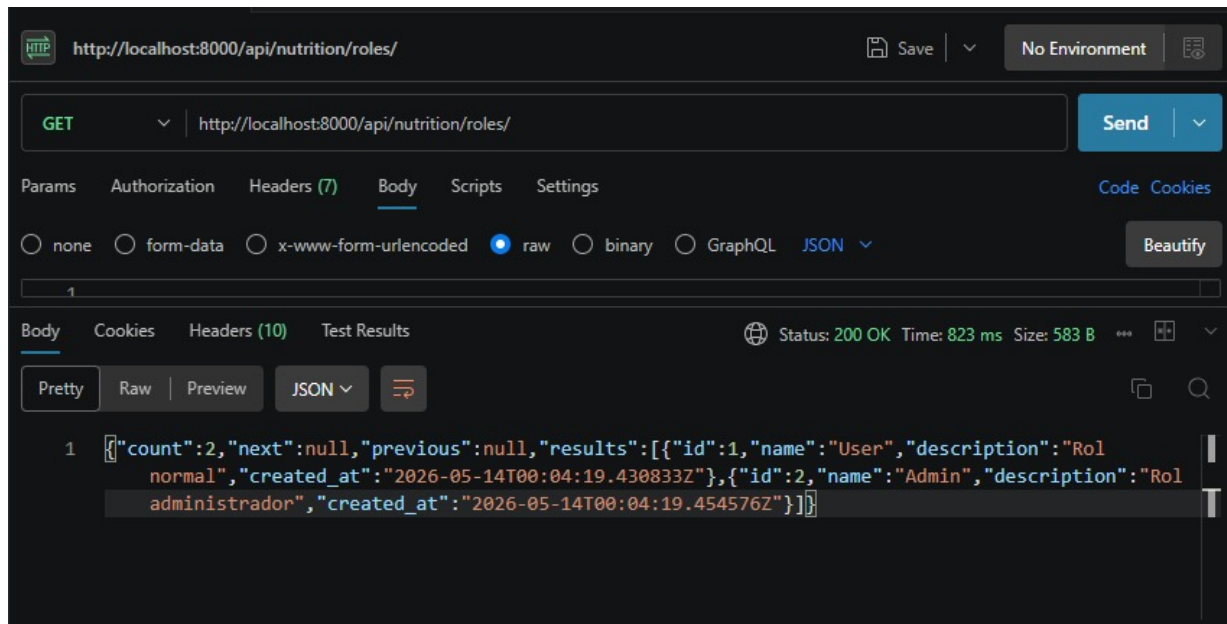


Figura 3: Respuesta exitosa de nuestra API en Postman